Concept 3: Hexagonal Architecture (Ports & Adapters)

1) Why people talk about Hexagonal Architecture

When you learn software architecture, you often start with the **three-tier model** *(three-tier architecture [architecture 3 couches])*:

1. **Presentation layer** *(presentation layer [couche présentation])*

   What users interact with: a UI, a frontend, or calling an API.

2. **Logic layer** *(business logic [logique métier])*

   Where the "real rules" of the application live: computations, decisions, workflows.

3. **Data layer** *(data layer [couche données])*

   Where data is stored and retrieved: database, files, etc.

This model is a good start, but it's **easy to make layers highly coupled** *(coupling [couplage])*—meaning one layer "knows too much" about another and becomes hard to change.

People use things like **dependency injection** *(dependency injection [injection de dépendances])* and **abstract classes** *(abstract classes [classes abstraites])* to reduce coupling, but **hexagonal architecture goes even further** in decoupling.

---

2) Other name: Ports and Adapters

Hexagonal architecture is also known as the **Ports and Adapters pattern** *(ports and adapters [ports et adaptateurs])*.

The key observation from Alistair Cockburn (who popularized it) is:

Interacting with a **database** *(database [base de données])* is not that different from interacting with **external applications** *(external systems [systèmes externes])*.

They all follow a similar pattern:

- your app needs a way to talk to “something outside”
- but your core logic shouldn’t be tied to how that thing works.

3) The Hexagon mental model

Imagine your application as a **hexagon** *(hexagon [hexagone])*:

- The **center** is your **core logic** *(core domain logic [logique métier centrale / domaine])*.
- Around it, you have multiple **inputs** *(inputs [entrées])* and **outputs** *(outputs [sorties])* to the outside world.

Hexagonal doesn’t mean “6 only”. It’s a picture to show **many sides**.

---

4) Ports: the contracts your application defines

For **every input and output**, you create a **port** *(port [port])*.

A **port** is an **abstraction** *(abstraction [abstraction])*—basically a **contract** *(contract [contrat])* defined by your application that says:

> “This is how the outside world can interact with me”
>
> or
>
> “This is what I need from the outside world.”

- The app **does not care** where the data comes from or goes to:
  - database *(database [base de données])*
  - file system *(file system [système de fichiers])*
  - message queue *(message queue [file de messages])*
  - etc.

As long as the port contract is satisfied, the core logic stays the same.

Important nuance from the video about interfaces

We often implement ports using **interfaces** *(interface [interface])*.

But the video warns about a common mistake:

People create interfaces like `IDbRepository` *(repository [dépôt / repository])* which already assume the technology ("Db" = database).

Hexagonal thinking says: the core logic should not "care" that it's a database.

So the port should represent what the app needs conceptually (read/write), not the technology.

5) Adapters: the translators to real technology

Once you have a port, you implement it with an **adapter** *(adapter [adaptateur])*.

An **adapter** contains the "technology-specific" logic:

- converting the app's data to a database format *(mapping/translation [conversion / mapping])*
- calling APIs
- writing files
- sending messages, etc.

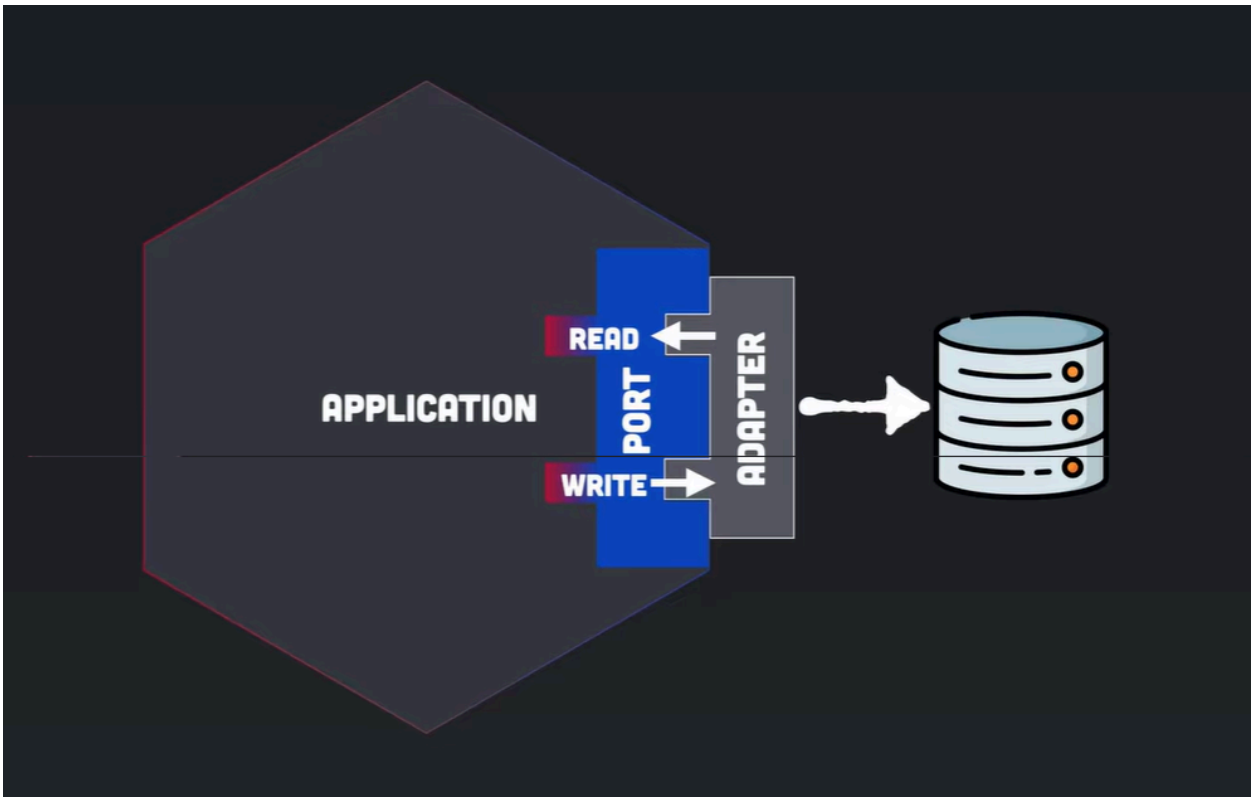
Think of the adapter as a **converter** *(converter [convertisseur])*:

- It takes what comes through the port (in your app's language),
- and converts it into what the external system expects.

Key promise:

You can change the adapter as much as you want, **and your core application + port do not change**.

That's the decoupling benefit.



7) Driving side vs Driven side

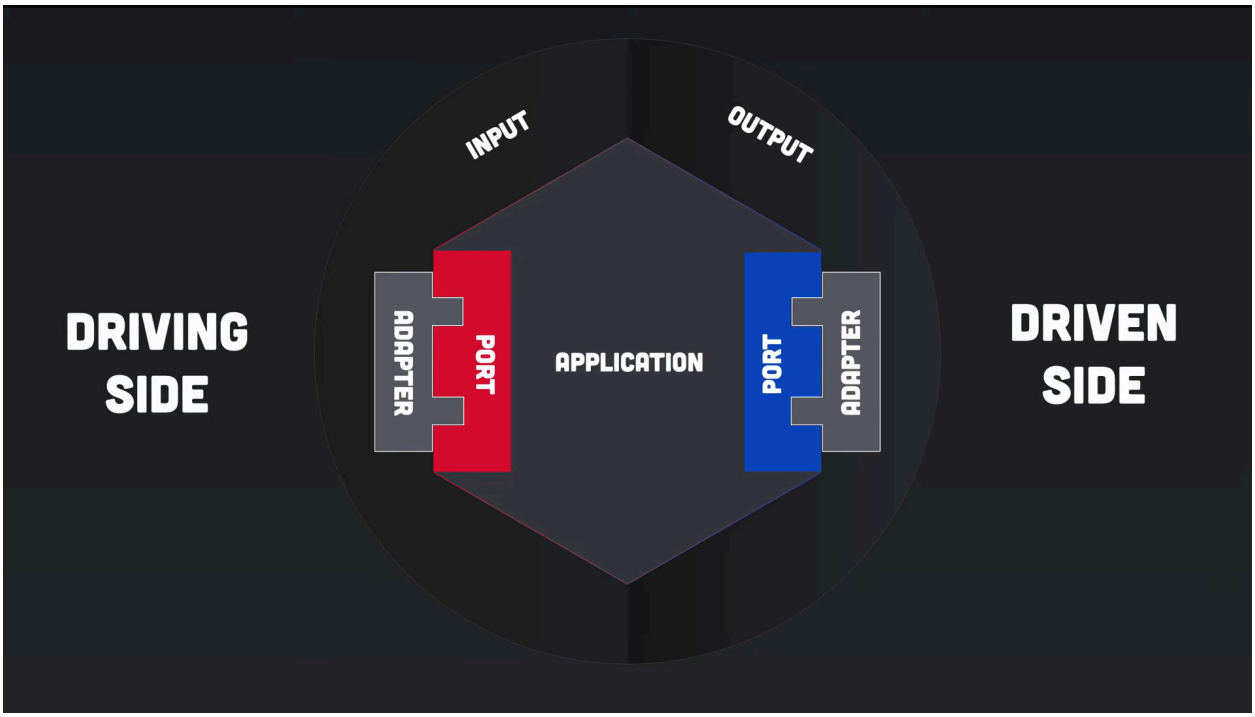
Alistair describes two sides:

- **Driving side** *(driving side [côté pilotant])* = inputs

  They “drive” the application to do something.

- **Driven side** *(driven side [côté piloté])* = outputs

  They are “driven” by the application (the app calls them).



10) Splitting a large system into multiple hexagons (DDD idea)

If an application is large, you can split it into multiple hexagons.

This matches the idea behind **Domain-Driven Design** *(DDD [conception pilotée par le domaine])*:

Each hexagon represents one **domain** *(domain [domaine])* and has a **single responsibility** *(single responsibility [responsabilité unique])*.

Examples the video mentions:

- user management *(user management [gestion des utilisateurs])*
- search *(search [recherche])*
- file persistence *(persistence [persistance])*
- email sending *(email sending [envoi d'emails])*

Each domain-hexagon can be an independent unit, and ports/adapters connect them together.

Yes — **every microservice (ms) can be built using hexagonal architecture**.

Think of it like this:

- **Microservice** = *the boundary of what is deployed separately* (a "box" on your architecture diagram)
- **Hexagonal architecture** = *the internal structure inside that box* (how you organize code to avoid coupling)

So you can absolutely do:

One microservice = one hexagon (most common)

Example:

- `rfq_manager_ms` is a hexagon:
  - **Core**: RFQ rules, stages, KPIs
  - **Ports**: `RFQRepository`, `NotificationPort`, `UserAccessPort`
  - **Adapters**: Postgres adapter, Kafka adapter, REST adapter, etc.

Or one microservice can contain multiple mini-hexagons (if it's big)

If `rfq_manager_ms` becomes huge, you might split inside it into sub-domains:

- RFQ
- Workflow/Stage
- KPI/Admin

Each with its own ports/adapters, still inside the same deployment.

Domain (center) — your business logic. "How do I create an RFQ? What makes an RFQ overdue? How do I calculate a deadline?" This never changes regardless of who's asking.

Ports (middle) — interfaces (like doors). They define *what* the domain can do, without knowing *who* is calling. Think of it as a menu of capabilities.

Adapters (outer) — the different ways to access the domain. Each adapter plugs into a port. A REST adapter handles HTTP requests from the front-end. A batch adapter runs on a schedule for reminders. Both use the **same** domain logic through the **same** ports.